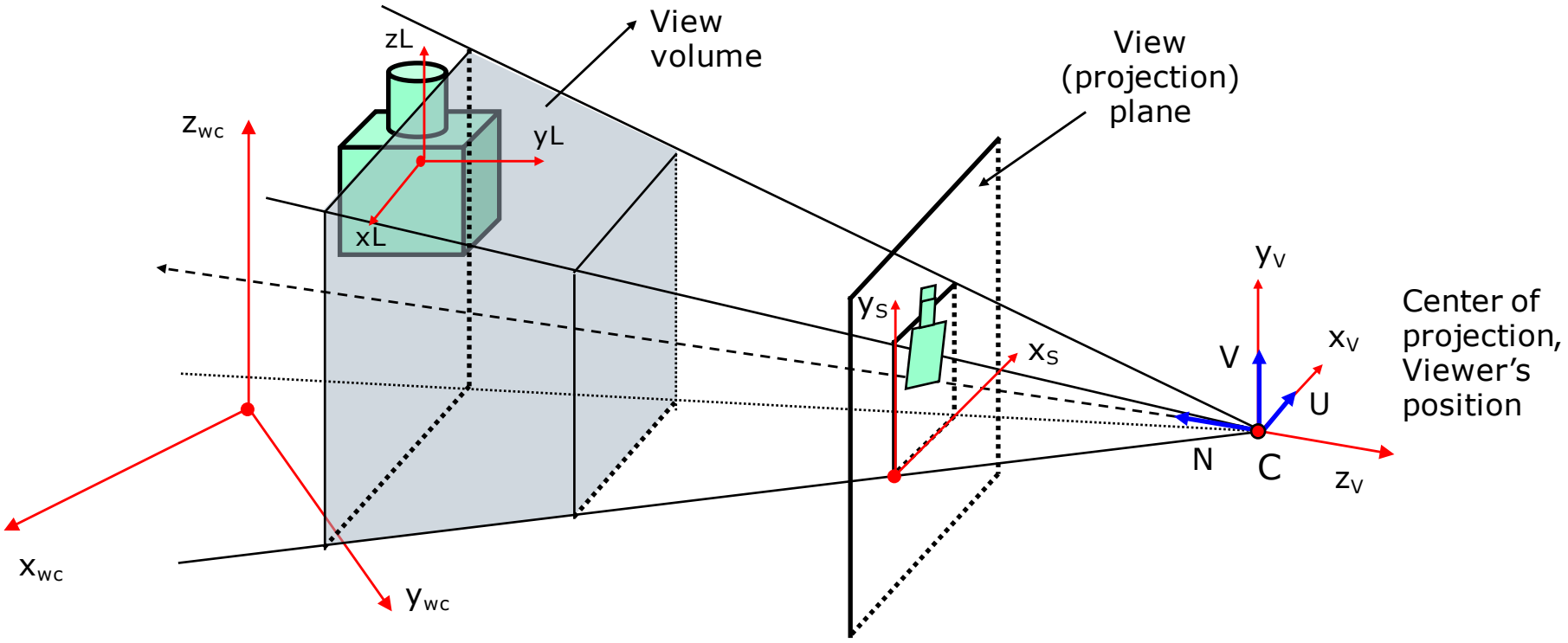
Polygonal Objects Rendering (2)

Contents

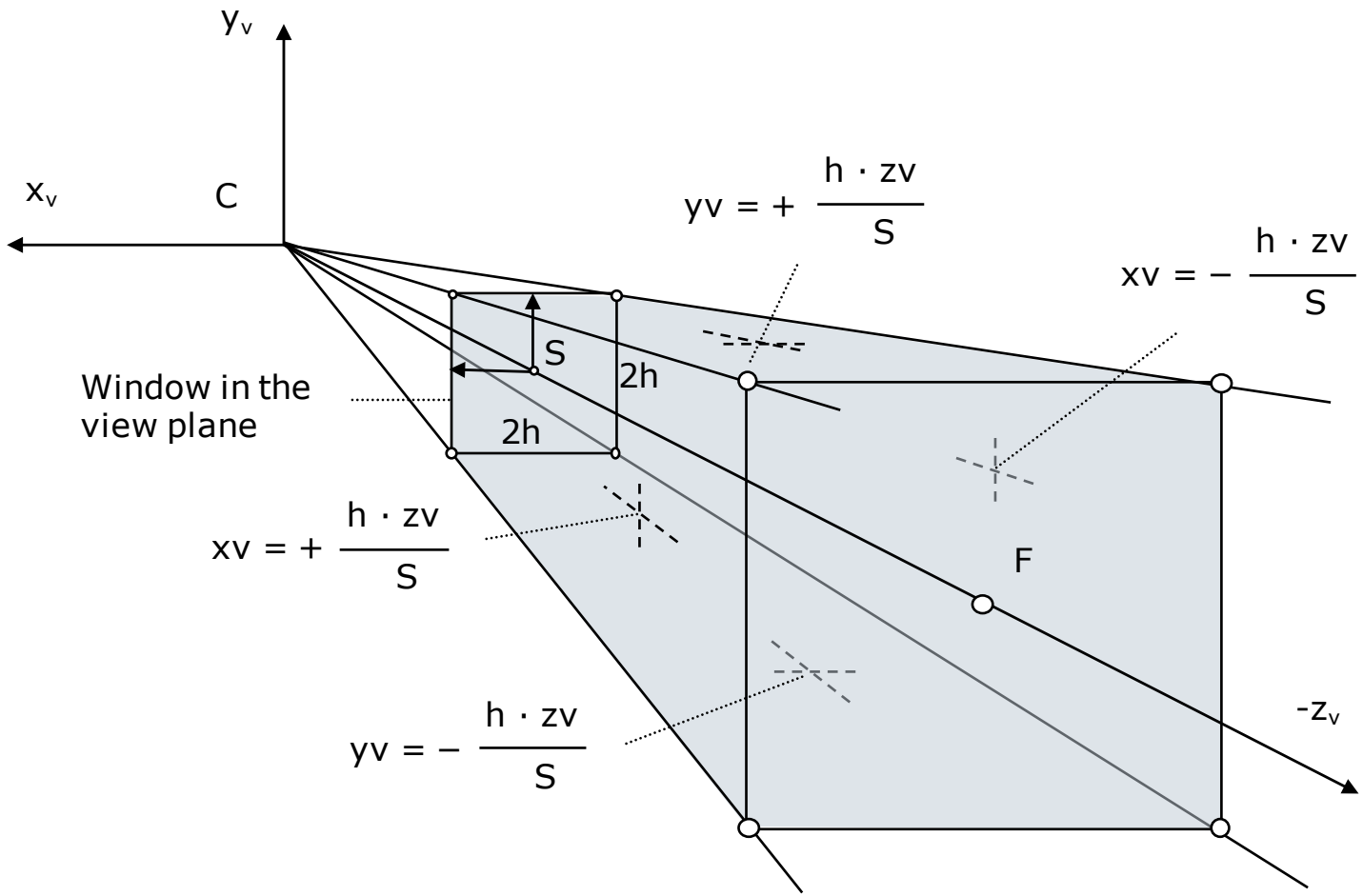
- Screen coordinate system
  - View volume
  - Transformation to screen space
  - 3D clipping against the view volume
  - Screen space rendering
    - Rasterization
    - Hidden surface removal
    - Incremental shading
      - Gouraud shading
      - Phong shading
    - Linear interpolator

Conceptual model of 3D viewing process



View volume

- Screen space is defined inside the view volume (viewing frustum)



View volume

- View volume definition: 6 planes

$$zv = S$$

$$zv = F$$

$$xv = \pm \frac{h \cdot zv}{S}$$

$$yv = \pm \frac{h \cdot zv}{S}$$

- 2h is the view plane window dimension
- S is the view plane distance, the virtual screen plane distance, and the near plane distance
- F is the far plane distance
- S and F are application based parameters $\pm$

Transformation to screen space

- Screen space is a 3D space
- Transformation conditions:
  1. zs needs to be normalized so that precision is maximized
  2. Lines in view space need to transform into lines in screen space
  3. Planes in view space need to transform into planes in screen space
- To satisfy the conditions the transformation of z takes the form:

  $$zs = A + B/zv$$

  where A and B are constants with the following constraints:

  1. Choosing $B<0$ so that as zv increases than so does zs. This preserves the intuitive notion of depth. If one point is behind another, then it will have a larger zv value. If $B>0$ it will also have a larger zs value
  2. Normalizing the range of zs values so that the range $zv \in [S, F]$ maps into the range $zs \in [0, -1]$.

     Obs. The normalization into the negative range [0, -1] is given by the localization of the scene of objects on the –zv axis, $S \leq 0$, $F \leq 0$

Transformation to screen space

Perspective transformation is given by the normalization relation:

$$\frac{h \cdot zv}{S} \rightarrow 1 \quad \text{then} \quad xv \rightarrow xs$$

therefore the screen coordinate values are:

$$xs = xv \cdot \frac{S}{h \cdot zv}$$

$$ys = yv \cdot \frac{S}{h \cdot zv}$$

Conditions on the near and far planes of the frustum give:

$$A + B / S = 0$$

$$A + B / F = -1$$

and results $A = F / (S - F)$, $B = - F S / (S - F)$

Transformation to screen space

Finally the transformation is given by the relations:

$$xs = S \cdot \frac{xv}{h \cdot zv}$$

$$ys = S \cdot \frac{yv}{h \cdot zv}$$

$$zs = F \cdot \frac{1 - S / zv}{S - F}$$

the additional constant h ensures that xs and ys fall in the range [-1,1].

The transformation is not linear and cannot be expressed as a matrix. For that reason it is expressed in homogeneous coordinates and split in two parts: (1) linear part, and (2) non-linear part.

(1) Linear transformation: $X = xv$

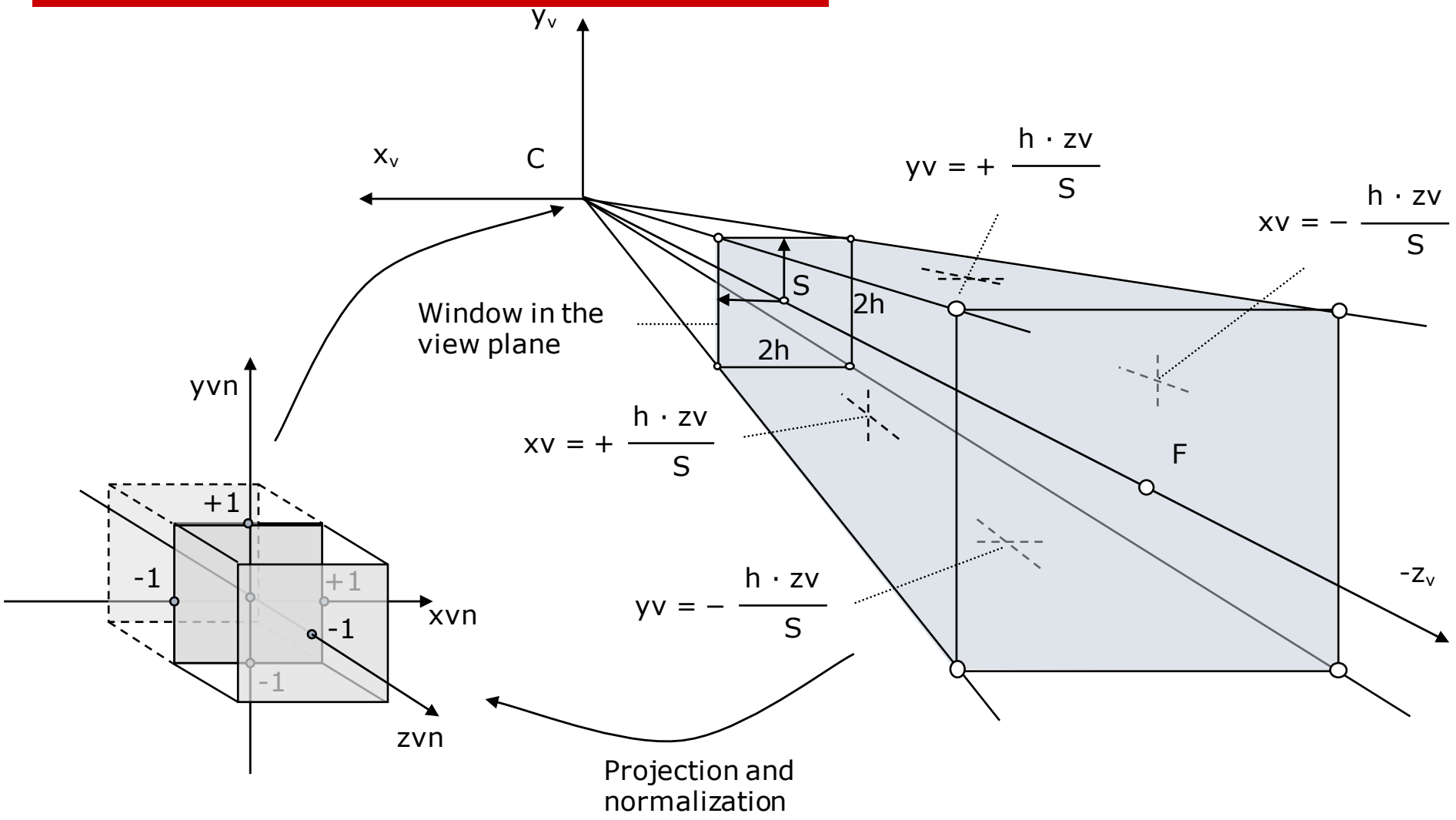
$Y = yv$

$$Z = \frac{h \cdot F \cdot zv}{S\,(S - F)} - \frac{h \cdot F}{S - F}, \text{ and } w = \frac{h \cdot zv}{S}$$

(2) Non-linear transformation: perspective division by w. The final screen coordinates are:

$xs=X/w$, $ys=Y/w$, $zs=Z/w$

Screen space normalization



Transformation to screen space

- The coordinates (X, Y, Z, w) are called homogeneous coordinates. The fourth coordinate w keeps the depth information, required for later graphics operations (e.g. hidden surface removal)
- The reason for that separation is to express the linear perspective transformation as a 4x4 matrix (i.e. T_{pers})
- The perspective transformation matrix can be concatenated with other previous transformation matrices in order to define a sequence of transformations called **rendering pipeline**

$$\begin{bmatrix} X & Y & Z & w \end{bmatrix} = \begin{bmatrix} xv & yv & zv & 1 \end{bmatrix} T_{pers}$$

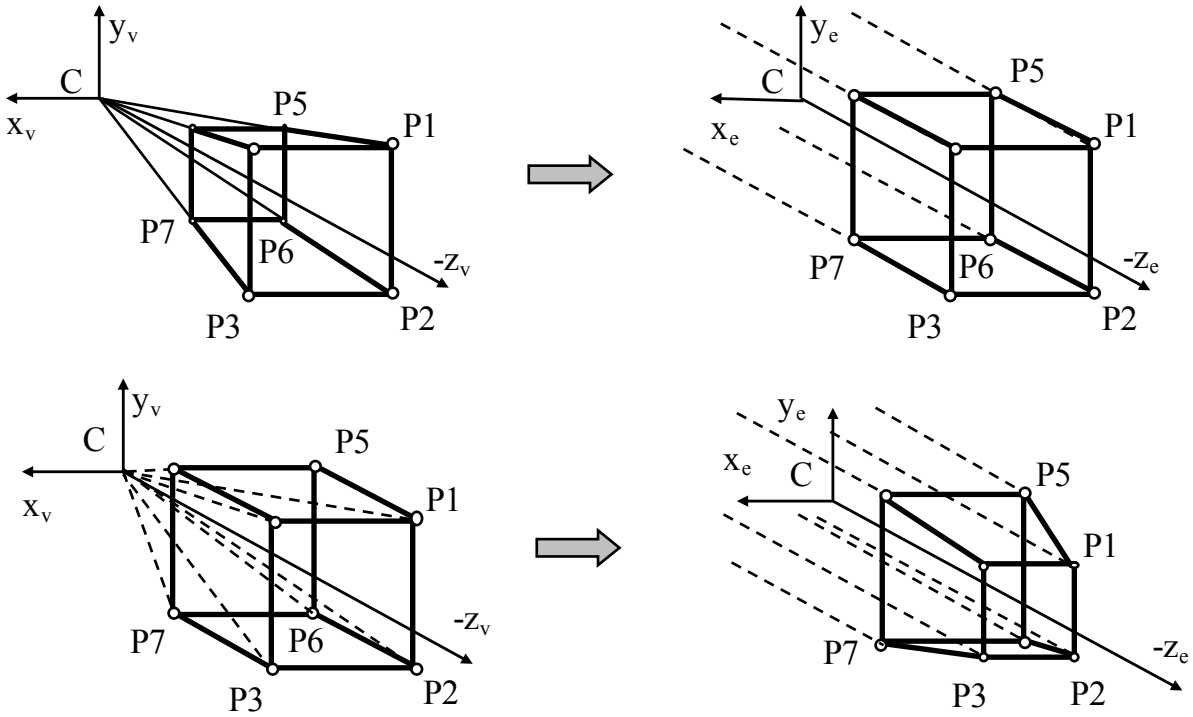
$$T_{pers} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & \frac{h \cdot F}{S\,(S - F)} & \frac{h}{S} \\ 0 & 0 & -\frac{h \cdot F}{S - F} & 0 \end{bmatrix}$$

Transformation to screen space

- Perspective transformation is the unique transformation that really uses the w coordinate. The homogeneous representation of the Euclidian space is imposed by the need of the perspective in the rendering pipeline
- The standard transformation matrices Mij such as scale, rotation and translation have the element M33 always equal 1. For these transformations w is redundant. W is always 1 and remains 1 even after the transformation.
- Through the transformation from the view space to the screen space the equal distances on Zv are transformed into non-equal distances on Zs. Anyway, both coordinates keep depth information, that is distance on Z.
- For non-equal distances the correspondent interpolation operation in view space and screen space give different results.
- The objects are deformed toward the back size of the frustum.
- The hidden surface removal algorithms are more performant in the screen space than in world or view coordinate systems.

Transformation to screen space

- In screen space the view rays are parallel with Zs axis, because the projection center moves to infinity, along the Zs axis (if zv = 0, then zs -> - ∞). Therefore the computation of the hidden points means to compare the coordinates xs, and ys respectively.
- A frustum in the view space becomes a cube in the screen space, and similarly a cube becomes a frustum in the screen space.



3D clipping against the view volume

- Motivation for clipping the objects against the view volume before transformation from the view space to the screen space:
  1. Application needs to deal only with objects that contribute to the final image (i.e. inside the viewing frustum)
  2. The screen space transformation is not defined outside the viewing frustum. There is an exception for zv=0
- An object can have the following position relative to the viewing frustum:
  1. Completely outside the viewing frustum – it is removed from the model
  2. Completely inside the viewing frustum – it is transformed into the screen space and rendered
  3. Intersects the viewing frustum – it is clipped against the frustum and then transformed into the screen space

3D clipping against the view volume

- The clipping conditions are given by:

  $-1 \leq xs=X/w \leq 1$

  $-1 \leq ys=Y/w \leq 1$

  $-1 \leq zs=Z/w \leq 0$, where $w=(h \cdot zv)/S \geq 0$

  or the cliping limits:

  $-w \leq X \leq w$

  $-w \leq Y \leq w$

  $-w \leq Z \leq 0$

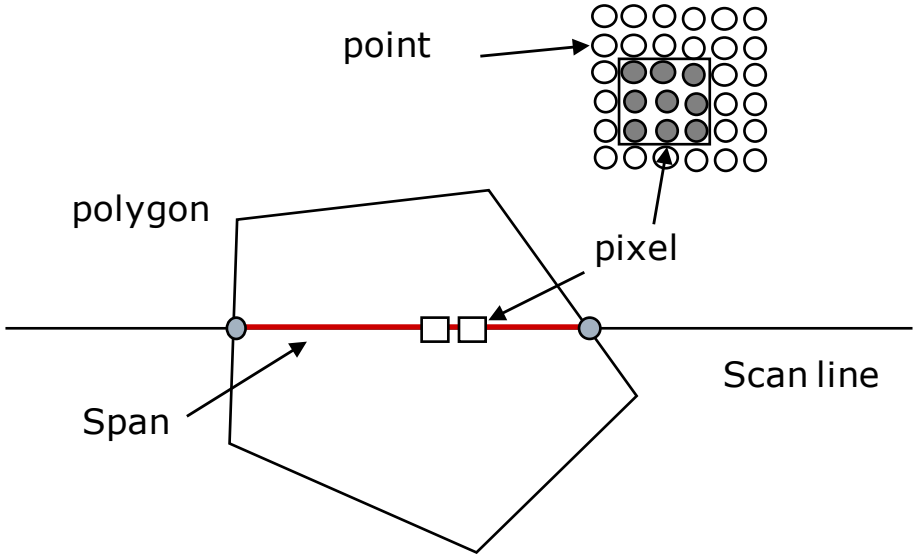
- The clipping operation can be performed on the homogeneous coordinates just before the perspective division

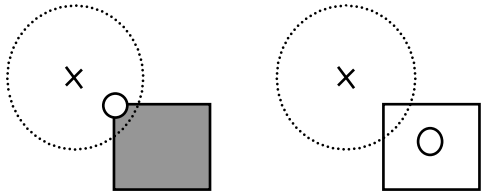
Screen space rendering

1. Rasterization
   - assigns a value to each pixel in the projection.
2. Hidden surface removal
   - removes invisible parts of polygons
3. Shading
   - compares the orientation of each polygon with the light source direction and assigns a shade to the interior of the polygon
4. Anti-aliasing
   - computes intermediate values for the overlapped projected parts of the polygons

Rasterization

- Scan line algorithm



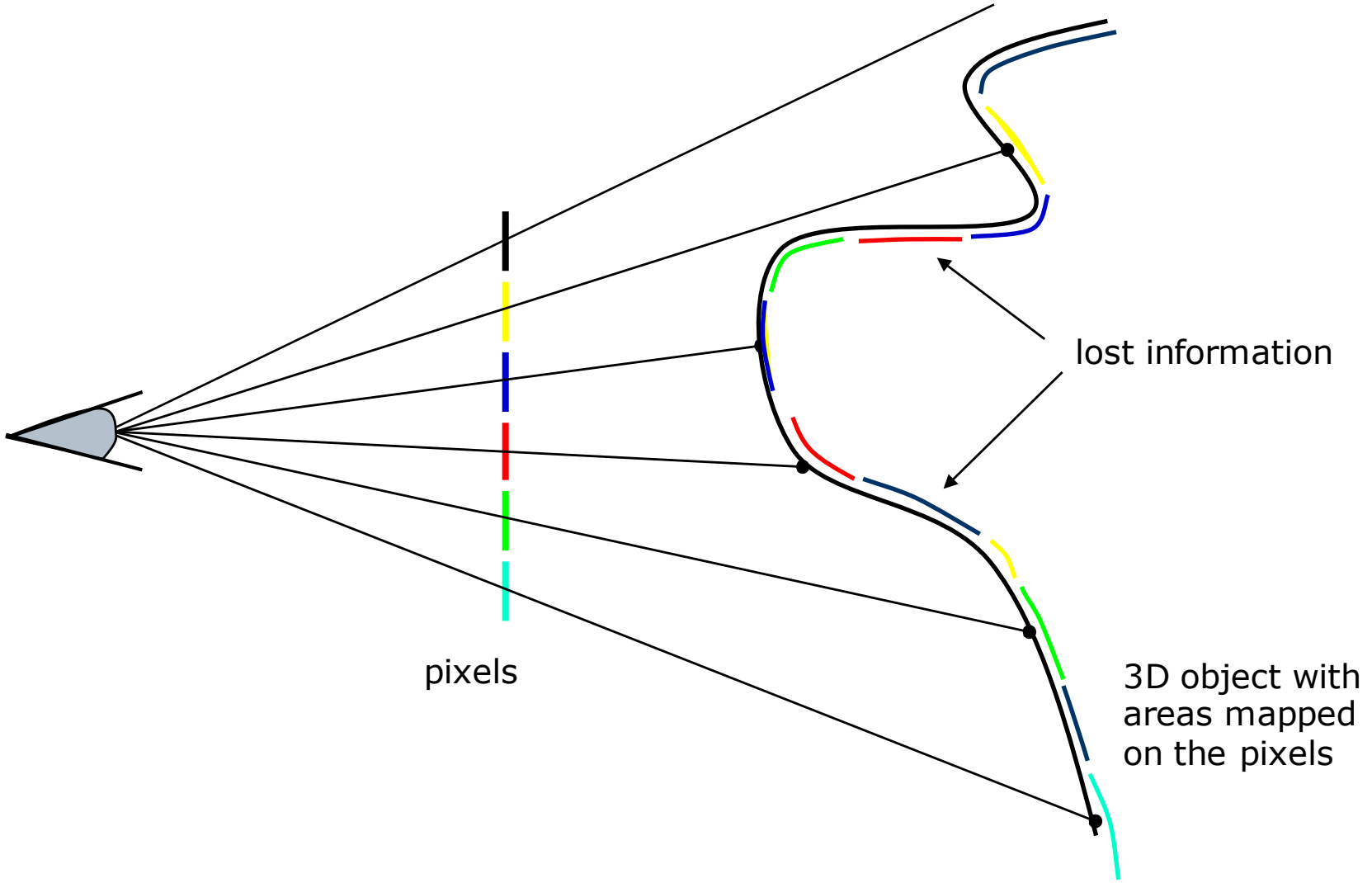


○ Pixel position

x Object point position

Relationship between pixel position and object point.

Mapping points into pixel



Hidden surface removal

- Z-Buffer algorithm (Catmull, 1975)

- Advantages of the Z-Buffer algorithm:
  - Simple implementation
  - Processes the polygon based models
  - Does not impose any pre-processing of polygons
  - Processes the polygons in any order they are fetched from the graphics model

- Disadvantages of the Z-Buffer algorithm:
  - Shading computation performed also for the pixels that are to be hidden by others
  - Needs large memory resources
  - Computation time increases in the case of the perspective projection
  - The size of the Z-Buffer depends on the accuracy to which the depth value of each point (x,y) is to be stored (e.g. integer vs floating point representation of the coordinates)

Incremental shading techniques

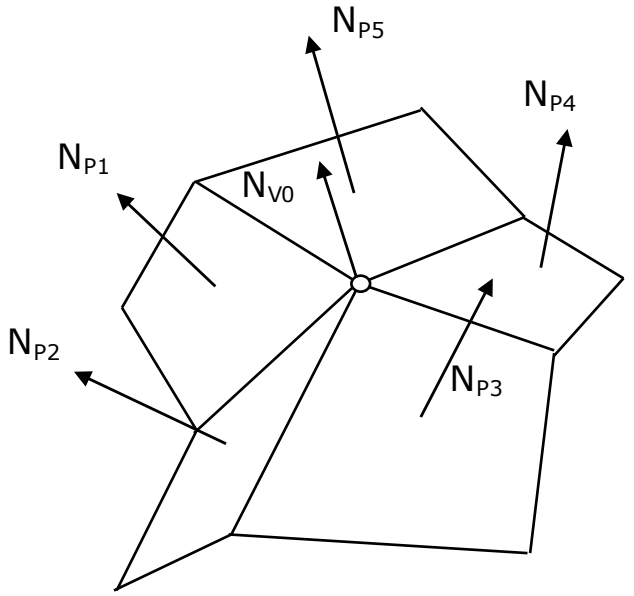
- Compute the light intensity and color over a polygon surface. There are two basic techniques:
  1. Gouraud interpolation (Gouraud, 1971)
  2. Phong interpolation (Phong, 1975)

- Characteristics:
  - Phong interpolation gives more accurate highlights and is generally preferred model
  - Gouraud method tends to be confined to applications where the diffuse component is sufficient, or to preview images prior to final rendering
  - Phong shading is more expensive than Gouraud shading. Phong calculation can exceed more than 50% of the total rendering time
- Issues
  - Final polygon visibility – polygon boundaries should be invisible in the final shaded. There two functions of a shading scheme for polygon meshes:
    1. Use some interpolative method for interior points
    2. Diminish the visibility of the polygon mesh approximation

Gouraud shading

- Gouraud interpolation (Henri Gouraud, 1971)
  - Gouraud, H.: "Continuous Shading of Curved Surfaces", IEEE Trans. On Computers, C-20(6), June 1971, pp.623-629.

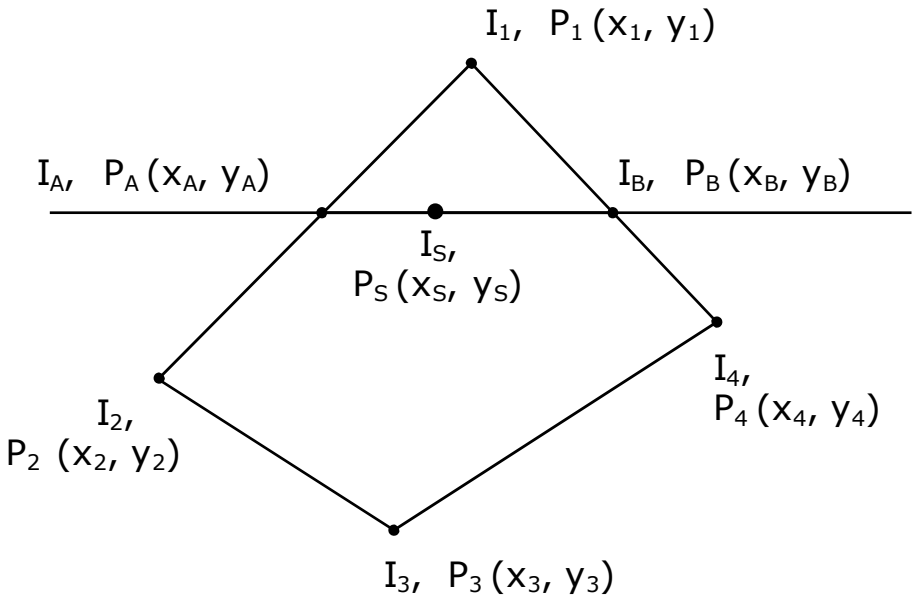
- Computes the light intensity for points inside a polygon
- Consider the light source at infinity
- Each vertex has associated a normal vector that is supposed to be orthogonal to the initial curved surface
- The vertex normal could be given or computed from the normals of the neighbor polygons



$$N_{V0} = 1/n\ (\alpha_1 N_{P1} + \alpha_2 N_{P2} + \ldots + \alpha_n N_{Pn})$$

Gouraud shading

- Algorithm steps:
  1. Compute light intensity I_V for each vertex. Light intensity is a function of normal vector in the vertex and its relative position to the light source and the viewer's position
  2. For each scan line position compute by interpolation the light intensities I_A and I_B on the span end points A and B. The span is given by the intersection between the scan line and two edges of the polygon

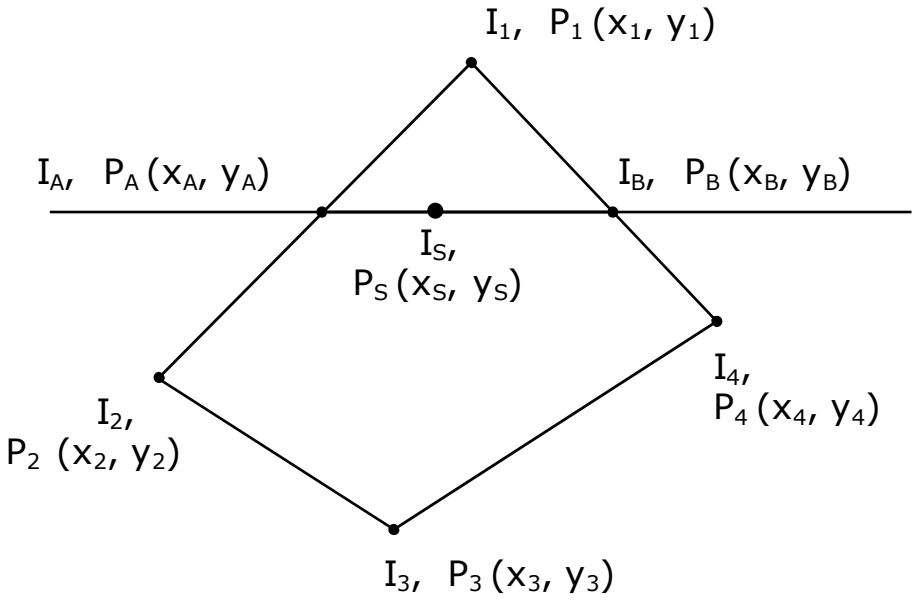


Gouraud shading

- Algorithm steps (continuation):

  3. For each point position S on the current span compute by interpolation the light intensity I_S from the end point intensities I_A and I_B

$$I_A = I_1 + (I_2 - I_1) / (y_2 - y_1) \cdot (y_A - y_1)$$
$$I_B = I_1 + (I_4 - I_1) / (y_4 - y_1) \cdot (y_B - y_1)$$
$$I_S = I_A + (I_B - I_A) / (x_B - x_A) \cdot (x_S - x_A)$$

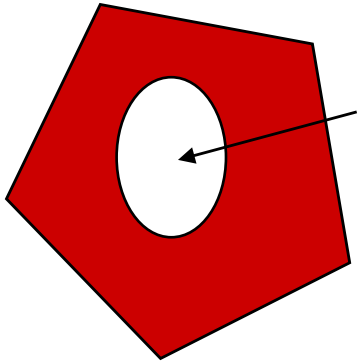


Gouraud shading

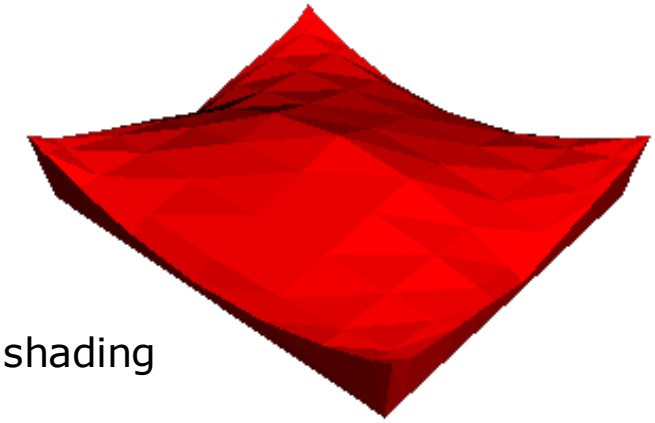
- The highlight is not visible by the Gouraud shading.

Some examples from the course Computer Graphics, by Pascal Vuylsteker, AN Univ., 2003.

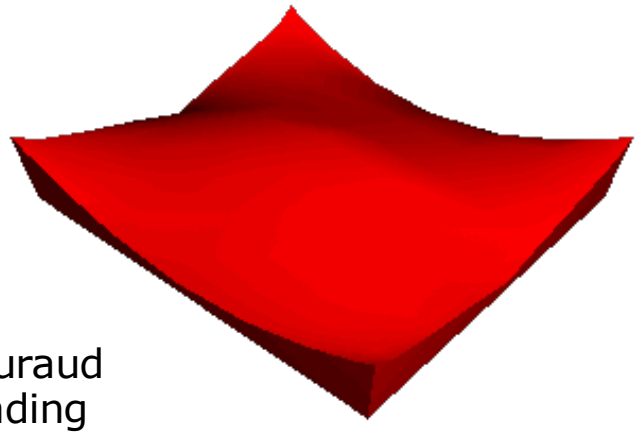
Flat shading



Light patch non-visible by the Gouraud shading



Flat shading

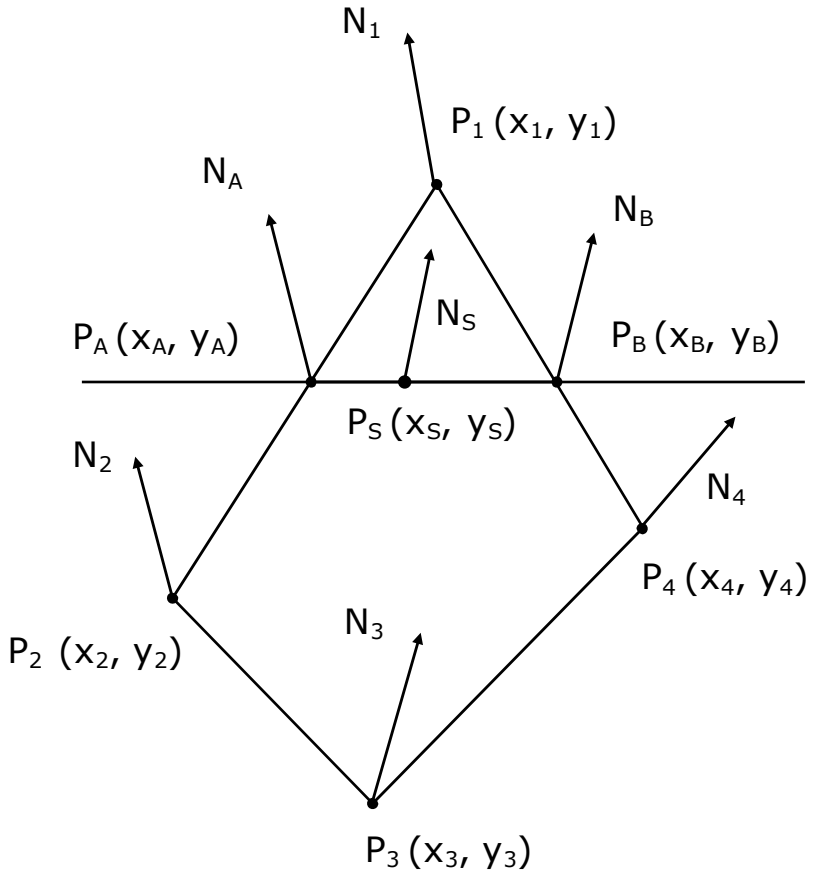


Gouraud shading

Phong shading

- Phong interpolation (Bui-Tuong Phong, 1975)
  - Phong, B. :"Illumination for computer-generated pictures", Comm. ACM, 18 (6), pp.311-317, 1975.

- Consider the light source at infinity
- Computes the light intensity for points inside a polygon
- The attributes interpolated are the vertex normals, rather than vertex intensities
- Each vertex has associated a normal vector that is supposed to be orthogonal to the initial curved surface
- The vertex normal could be given or computed from the normals of the neighbor polygons



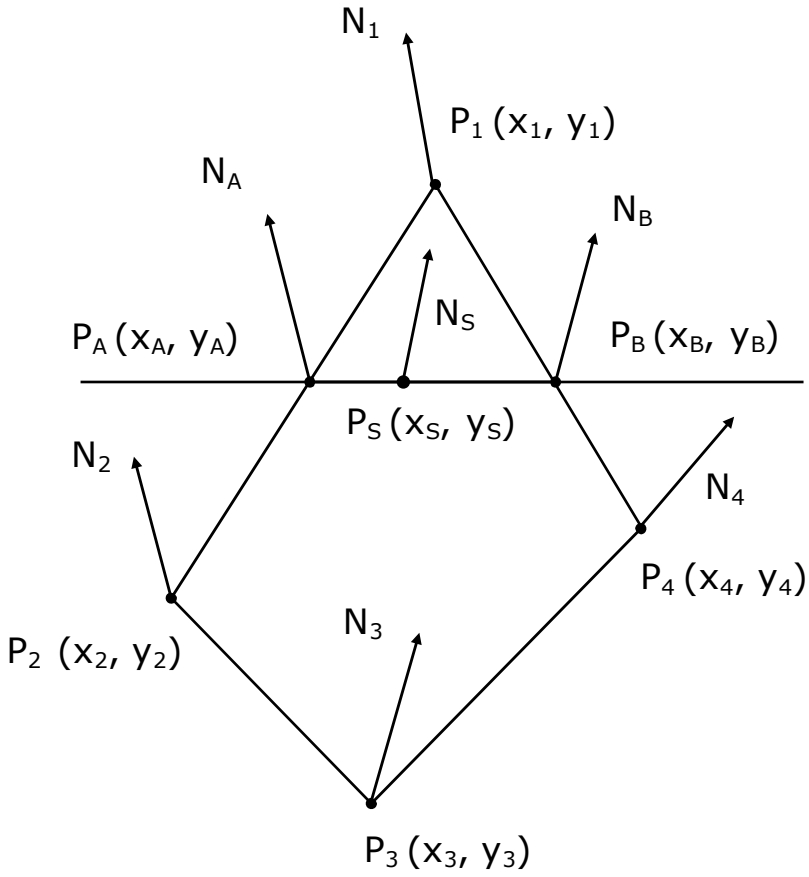
Phong shading

- Algorithm steps:

1. Compute the normal vector N_V for each vertex.
2. For each scan line position compute by interpolation the normal vectors N_A and N_B at the span end points A and B. The span is given by the intersection between the scan line and two edges of the polygon
3. N_A and N_B are used to interpolate N_S. By this approach a normal vector is computed for each point or pixel on the polygon:

$$N_{Sx}, N_{Sy}, N_{Sz}$$

1. Compute the light intensity using the N_S normal vector



Phong shading

$$N_A = N_1 + (N_2 - N_1)\frac{y_A - y_1}{y_2 - y_1}$$

$$N_B = N_1 + (N_4 - N_1)\frac{y_B - y_1}{y_4 - y_1}$$

$$N_S = N_A + (N_B - N_A)\frac{x_S - x_A}{x_B - x_A}$$

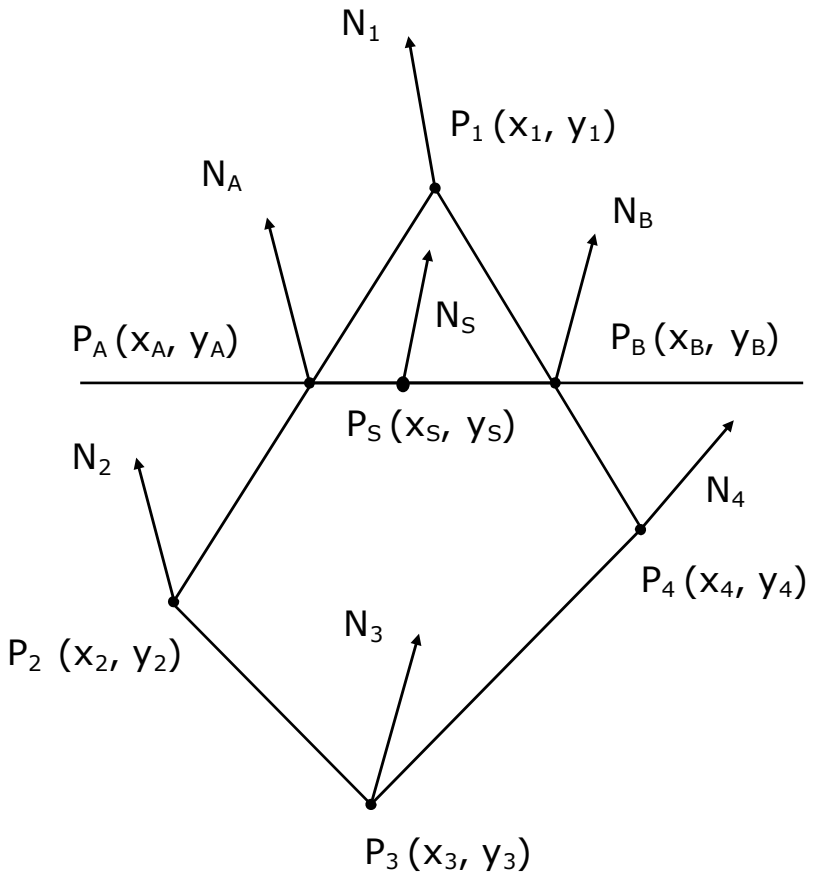
Incremental computation:

$$N_{Si} - N_A = (N_B - N_A)\frac{x_{Si} - x_A}{x_B - x_A}$$

$$N_{Si-1} - N_A = (N_B - N_A)\frac{x_{Si-1} - x_A}{x_B - x_A}$$

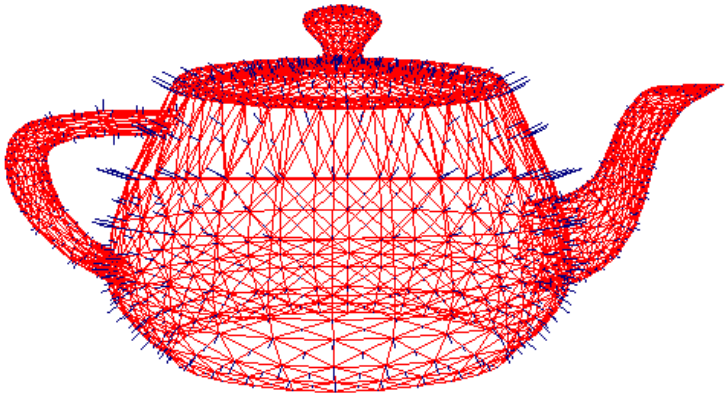
$$\Delta N_{Si} = \Delta x_{Si}\frac{\Delta N_{AB}}{\Delta x_{AB}}$$

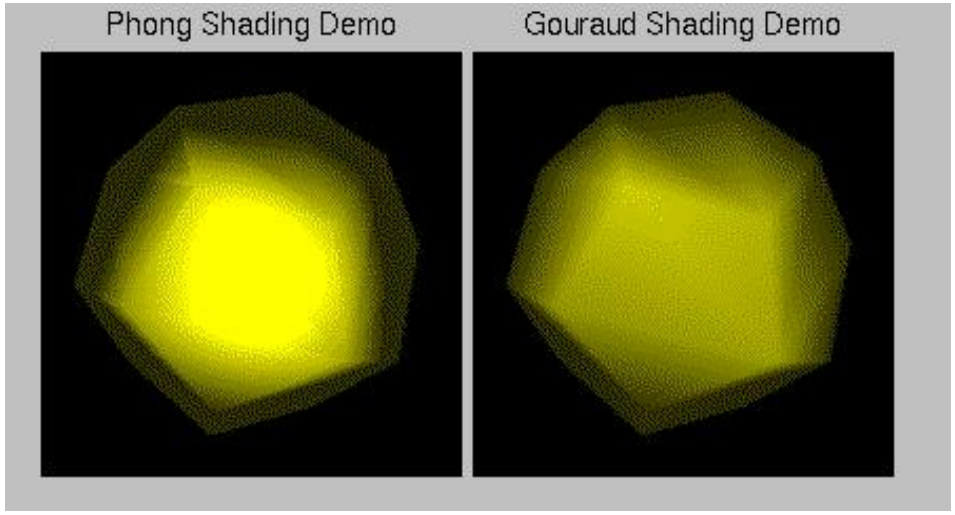
if $\Delta x_{Si} = 1$, then $\quad N_{Si} = N_{si-1} + \frac{\Delta N_{AB}}{\Delta x_{AB}}$

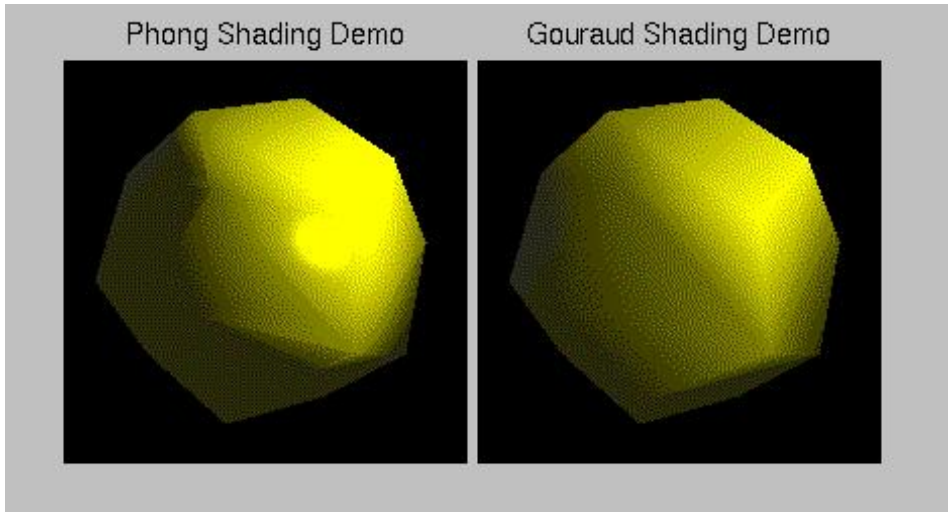


Phong shading

Some examples from the course Computer Graphics,
by Pascal Vuylsteker, AN Univ., 2003.





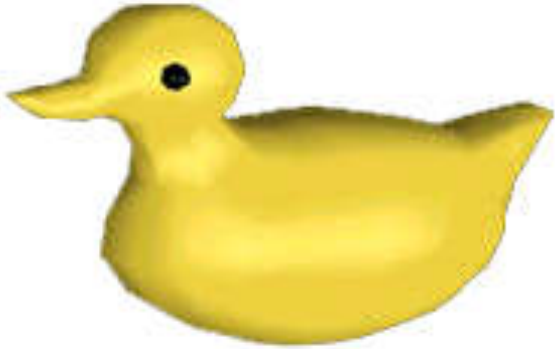


Gouraud and Phong shading

Examples taken from the course Computer Graphics, by Pascal Vuylsteker, AN Univ., 2003.

Flat shading

Gouraud shading



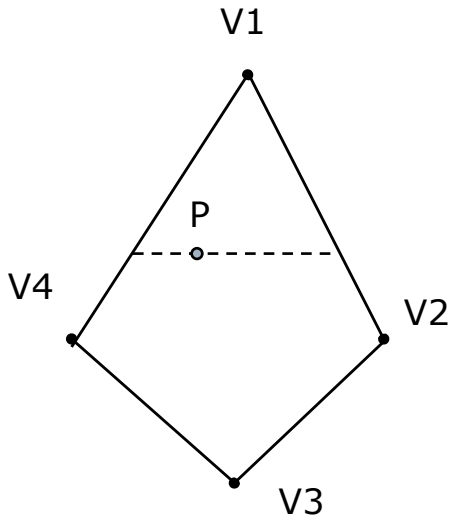
Phong shading

Gouraud and Phong shading

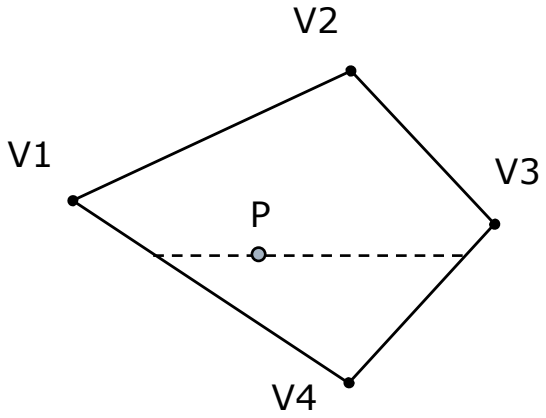
- Issues:
  1. Phong shading is three times as expensive as Gouraud interpolation, N_{Sx}, N_{Sy}, N_{Sz} against I_S
  2. Phong shading applies the local reflection model (e.g. Phong model) at each point, against the Gouraud shading that computes the light intensity just on the vertices
  3. Hard edges, high quality for great number of polygons
  4. Anomalies for rotation, figure (a) and (b)
  5. Light intensity computation depends on the underlying polygons, figure (c)
  6. Could give identical vertex normals from non-identical surface normals, figure (d)
  7. Anomalies because the intensity interpolation is carried out in screen coordinates from vertex normals computed in world coordinates, figure (e) and (f)

Issues in Gouraud and Phong shading

Visualization depends on object orientation.



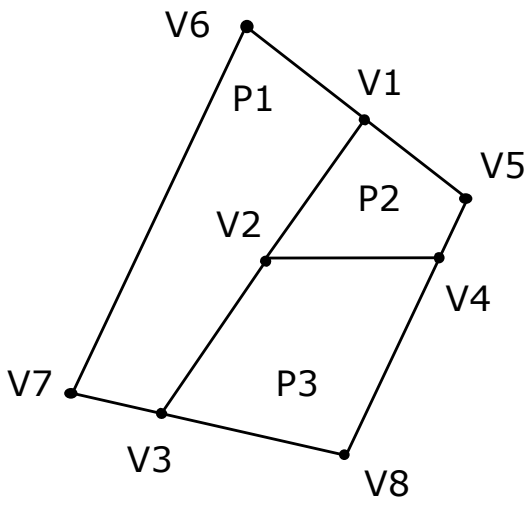
(a)



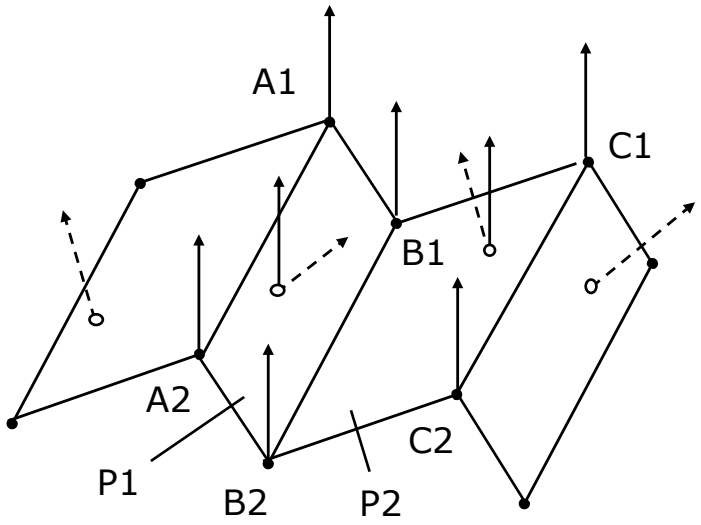
(b)

Issues in Gouraud and Phong shading

Visualization depends on the polygonal model (c). Initial vertex normal vectors could give unreal shading (d).



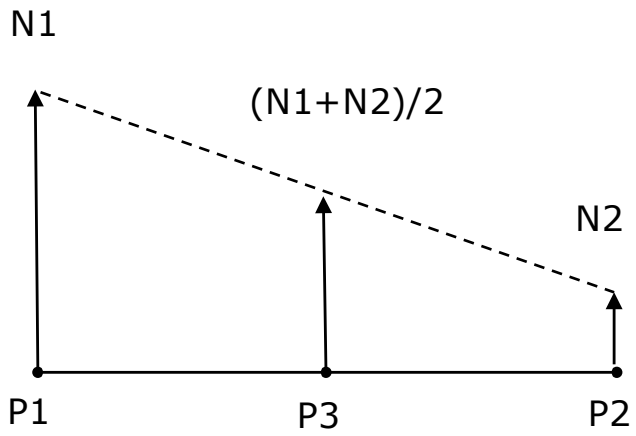
(c)



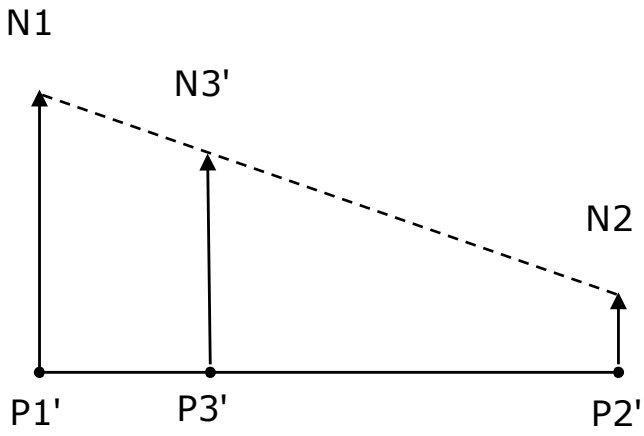
(d)

Issues in Gouraud and Phong shading

The computations in view space and screen space give different light intensity for corresponding points (P3 and P3’), for the distortion of distances and object shapes.



(e)



(f)

Speeding up Phong shading

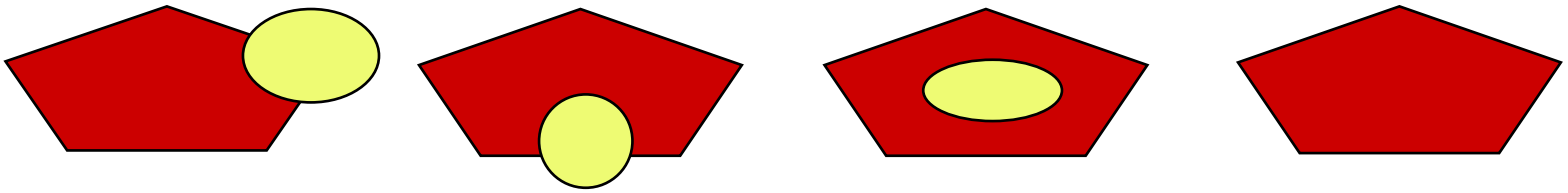
- Geometric techniques
  1. Align the light source (if it is unique) along the z axis
  2. Compute Phong shading for n^{th} point or pixel (e.g. n=2 or 4), and Gouraud shading (linear interpolation for light intensities) for the intermediate points
  3. Use the Phong shading just for polygons that involves highlights, and the Gouraud shading for all others:

     Highlight detection $N \cdot H \geq T$, where

     N is the normal vector at vertex or on the polygon edge;

     H = (L+V)/2, L – light direction vector, V – viewing vector (see the course on Illumination Models);

     T is a given light threshold

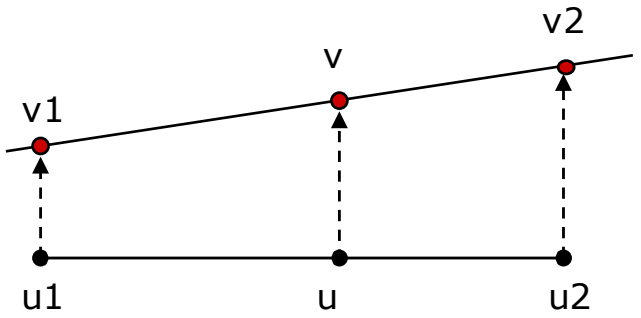


- Numerical techniques
  1. Evaluate the ambient, diffuse and specular term for each pixel with additions and memory access for pixel

Linear interpolator

- Compute intermediate values along a span, for given end point values
- Applied for
  - Intersections between current scan line and polygon edges
  - Z distance in hidden line and surface removal algorithms
  - Light intensity through Gouraud shading
  - Normal through Phong shading

- Unique interpolator $\Delta v = \frac{v2 - v1}{u2 - u1} \Delta u$

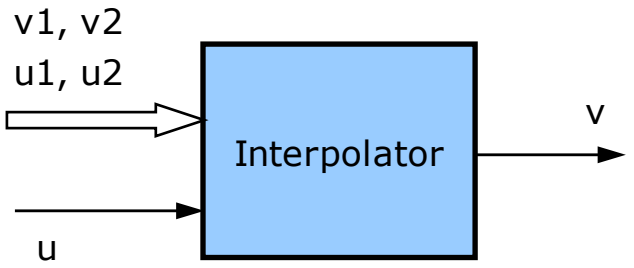


where

$\Delta v = v - v1$, $\Delta u = u - u1$

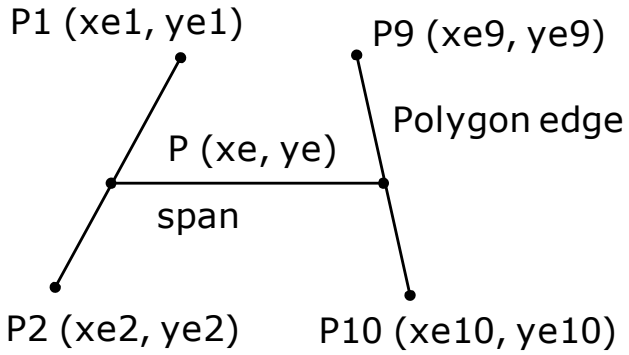
Inputs are v1, v2, u2, u1 şi u

Output is v

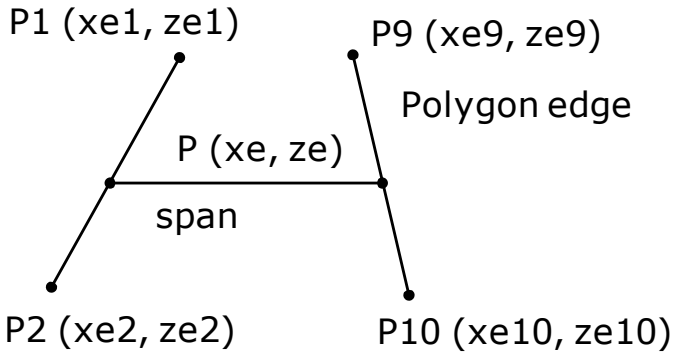


Linear interpolator

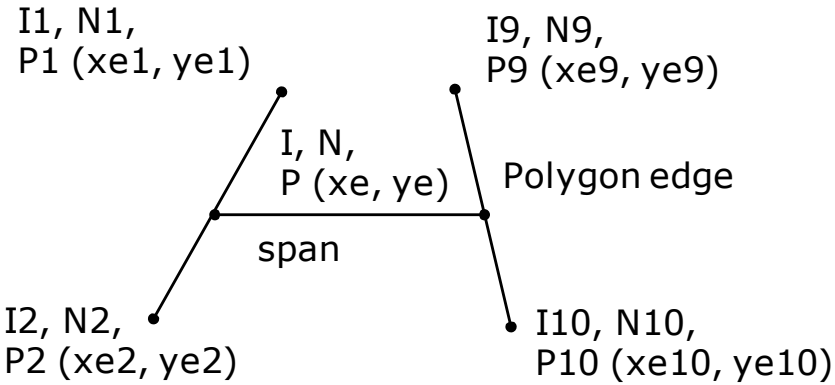
- Various inputs
  - Vertex coordinates (fig. a)
  - Vertex depth (fig. b)
  - Vertex light intensity (fig. c)
  - Vertex normal vector



(a). v = x, u = y



(b). v = z, u = x, or u = y



(c). v = I (Gouraud shading), or v = N
(Phong shading), and u = x, or u = y

Questions and problems

1. Explain where is located the projection plane within the viewing frustum?
2. Why the graphics model is defined in screen space rather than in screen plane?
3. Explain why the transformation from view space to screen space is performed through two phases?
4. What are the main advantages of the homogeneous coordinates?
5. Explain why the normalized screen space is a semi-cube?
6. Explain why the perspective projection from the view space is transformed into orthogonal projection in the screen space?
7. Exemplify the deformation of a cube, cylinder and sphere by transformation from the view space to the screen space.

Questions and problems

8. Exemplify the deformation of a rectangle, square, circle and triangle by transformation from the view space to the screen space.
9. Why the definition of the reference point for a pixel is important? How can be improved the graphics quality in such a case?
10. Why the z-Buffer algorithm can be applied in view and screen spaces but not in world coordinate space?
11. What is the main goal of the Gouraud and Phong shading algorithms?
12. Explain why and how the Gouraud and Phong shading algorithms works in screen space and screen plane? What are the differences than working similarly in view space and projection plane?

Questions and problems

13. Why the the Gouraud shading algorithm is not able to present light patches inside the polygon?
14. Explain why the Gouraud shading is less performant but faster than the Phong shading?